

Task 09 - Part 01: Question Answers

C# Fundamentals - Enums, Virtual/Sealed Members, Static, Operator Overloading, Equals/== & Generics

Q1. Why is it recommended to explicitly assign values to enum members in some cases?

By default, C# assigns enum members sequential integer values starting at 0. Explicitly assigning values is recommended whenever the numeric value carries real meaning outside the code - for example, if the enum values are stored in a database, sent over an API, or need to match an external specification/protocol. Explicit values also protect against bugs when members are reordered or new ones are inserted in the middle later: without explicit values, inserting a member shifts every subsequent member's underlying value, which can silently break any stored or serialized data that depended on the old values.

Q2. What happens if you assign a value to an enum member that exceeds the underlying type's range?

The code simply **fails to compile**. Every enum has an underlying integral type (int by default, or explicitly byte, short, long, etc.), and every member's value must fit within that type's valid range. Assigning a value outside that range - for example, 300 for an enum with an underlying byte type (max 255) - produces a compile-time error, since the compiler can determine this constraint statically before the program ever runs.

Q3. What is the purpose of the virtual keyword when used with properties?

Marking a property as virtual in a base class allows a derived class to **override** its get/set logic to provide different behavior, while still letting the property be accessed polymorphically through a base-class reference or variable. Without virtual, a property in a derived class can only be hidden (using new), not truly overridden, meaning the base implementation would always run when accessed through a base-typed reference regardless of the object's actual runtime type. virtual is what enables real polymorphic behavior for properties, exactly as it does for methods.

Q4. Why can't you override a sealed property or method?

The sealed keyword explicitly marks an already-overridden virtual member as the **final** implementation in the inheritance chain, preventing any further derived class from overriding it again. This is used when a class author wants to guarantee that a particular piece of behavior cannot be changed further down the hierarchy - for example, to protect critical logic from being altered in an unexpected way by a subclass, or to let the compiler safely make certain performance optimizations since it knows the implementation won't vary at that point in the hierarchy. Attempting to override a sealed member results in a compile-time error.

Q5. What is the key difference between static and object (instance) members?

A **static** member belongs to the **type itself**, not to any individual object - there is only one copy of it shared across the entire application, and it's accessed through the class name (e.g., `Utility.CalculatePerimeter(...)`) without ever needing to create an instance. An **instance (object) member** belongs to a specific object - each instance created from the class gets its own separate copy of instance fields, and instance members must be accessed through an object reference (e.g., `myObject.SomeMethod()`). Static members are appropriate for functionality or data that doesn't depend on any particular object's state, while instance members are for anything tied to an individual object's own data.

Q6. Can you overload all operators in C#? Explain why or why not.

No - C# only allows overloading a specific, predefined set of operators (such as `+`, `-`, `*`, `/`, `==`, `!=`, `<`, `>`, and a few others), and even for those, certain rules must be followed (e.g., `==` and `!=` must be overloaded together). Some operators **cannot** be overloaded at all, such as the assignment operator (`=`) itself, the conditional operator (`?:`), the null-coalescing assignment in certain contexts, and logical `&&`/`||` directly (though their underlying `&` and `|` can be overloaded, which indirectly affects short-circuit evaluation). This restriction exists to keep operator overloading predictable and prevent developers from redefining fundamental language semantics (like how assignment or control flow works) in ways that would make code dangerously unpredictable to read.

Q7. When should you consider changing the underlying type of an enum?

Changing an enum's underlying type (e.g., from the default `int` to `byte` or `short`) is worth considering when **memory efficiency** matters - specifically when the enum will be used in very large arrays, collections, or data structures where every byte saved per element adds up (an `int` is 4 bytes, while a `byte` is 1 byte). It's also relevant when interacting with external data formats, file layouts, or network protocols that expect a specific fixed-size representation for the value. As long as all of the enum's member values comfortably fit within the smaller type's range, shrinking the underlying type costs nothing functionally while reducing the memory footprint of the enum wherever it's stored.

Q8. Why can't a static class have instance constructors?

A static class can **never be instantiated** - there is no way to create an object of that type using `new`, since the whole point of a static class is to group static members (like a `Utility` class) without ever needing an instance. Since an instance constructor's entire purpose is to initialize a new object when one is created, and static classes forbid object creation entirely, allowing an instance constructor would be meaningless - there would never be an object for it to run against. A static class can only have a static constructor, which runs once, automatically, to initialize static data before the class is first used.

Q9. What are the advantages of using `Enum.TryParse` over direct parsing with `int.Parse`?

`Enum.TryParse()` is designed specifically to convert a string into an enum value **safely**: it returns a `bool` indicating success or failure instead of throwing an exception on invalid input, which means it can gracefully handle a string that doesn't correspond to any valid enum member (or isn't a

recognizable name/number at all) without crashing the program. Using `int.Parse()` directly only works if you already have the underlying numeric value as a string, and it does nothing to verify that the resulting number actually corresponds to a defined enum member - it happily parses "999" into an out-of-range enum value. `Enum.TryParse()` also supports parsing enum **names** directly (e.g., "Monday"), not just numbers, making it far more convenient and robust for handling real user input.

Q10. What is the difference between overriding Equals and == for object comparison in C# struct and class?

Equals() is a virtual instance method (inherited from `Object`) intended to define custom, value-based equality logic for a type - overriding it lets you specify exactly what makes two instances "equal" (e.g., comparing `Employee` objects by their `Id`). The `==` operator is separate and, by default, behaves differently depending on the type: for a class (reference type), `==` compares **references** (are they the same object?) unless explicitly overloaded to do otherwise; for a struct (value type), `==` is **not** defined by default at all and must be manually overloaded if you want to use it, since the compiler won't assume how to compare two struct instances' fields for you. Overriding `Equals()` does not automatically change what `==` does - they must be updated together (and typically `GetHashCode()` as well) to keep equality logic consistent across the type.

Q11. Why is overriding ToString beneficial when working with custom classes?

Without overriding it, `ToString()` just returns the object's fully-qualified type name, which isn't useful for understanding what a specific instance actually represents. Overriding it lets a class produce a clear, readable summary of its own state (e.g., an `Employee` printing as "John Doe - Engineering"), which is automatically used by `Console.WriteLine()`, string interpolation, and debugger displays. This makes debugging, logging, and simple diagnostic output far easier, since developers can immediately see meaningful information about an object without manually inspecting or printing each of its individual properties.

Q12. Can generics be constrained to specific types in C#? Provide an example.

Yes - C# supports **generic type constraints** using the `where` clause, which restricts what types can be used as a type argument for a generic type parameter. For example:

```
public static T Max<T>(T a, T b) where T : IComparable<T>
{
    return a.CompareTo(b) > 0 ? a : b;
}
```

Here, the constraint `where T : IComparable<T>` ensures that only types implementing `IComparable<T>` can be used with this method, which guarantees that `a.CompareTo(b)` is a valid call at compile time. Other common constraints include `where T : class` (reference types only), `where T : struct` (value types only), `where T : new()` (must have a public parameterless constructor), or `where T : SomeBaseClass` (must derive from a specific base type).

Q13. What are the key differences between generic methods and generic classes?

A **generic method** declares its own type parameter(s) on the method itself (e.g., `T Max<T>(T a, T b)`), and that type parameter only exists for the duration of that single method call - it can even live inside a completely non-generic class. A **generic class** declares its type parameter(s) at the class level (e.g., `class Stack<T>`), meaning the type is fixed for the entire lifetime of that instance - once you create a `Stack<int>`, every member of that particular instance works with `int`, and you cannot mix types within it. In short: a generic method's type parameter is scoped to one call, while a generic class's type parameter is scoped to the whole object.

Q14. Why might using a generic swap method be preferable to implementing custom methods for each type?

A generic `Swap<T>(ref T a, ref T b)` method can swap values of **any** type - `int`, `string`, a custom `Rectangle` struct, or anything else - using a single implementation, instead of writing and maintaining a separate, nearly identical `SwapInt`, `SwapString`, `SwapRectangle`, etc. for every type that needs swapping. This avoids code duplication, reduces the chance of bugs (since there's only one implementation to get right and maintain), and automatically supports any future type without requiring new code to be written - the generic version works for a type the moment it's compiled against it.

Q15. How can overriding Equals for the Department class improve the accuracy of searches?

By default, `Equals()` on a class compares references, meaning two `Department` objects representing the exact same department (e.g., both with `Name = "IT"`) would be considered "not equal" unless they happen to be the literal same object in memory. Overriding `Equals()` to compare meaningful data (like the department's name or ID) lets a search method such as `SearchArray` correctly find a matching `Department` based on its actual real-world identity, rather than requiring the exact same object reference to have been passed in. This makes searches behave the way a user would intuitively expect - matching by value, not by memory address.

Q16. Why is == not implemented by default for structs?

The compiler doesn't know, on its own, what "equal" should mean for an arbitrary custom struct - should two `Circle` structs be equal only if every field matches exactly, or only some of them (e.g., ignoring `Color` and comparing only `Radius`)? Because there's no single universally correct default behavior, C# does not automatically generate a `==` operator for structs the way it does provide a default (reference-based) `==` for classes. This forces the developer to explicitly define what equality means for that specific struct by overloading `==` (and typically `Equals()` and `GetHashCode()` alongside it) themselves, ensuring the comparison logic is intentional rather than an assumption made on the developer's behalf.